

Final Project – Bank Account Management

SQT1063 – Automation Testing

Jasprit Kaur (C0917419)

Amandeep Kaur (C0917405)

Rahul Shailesh Kawa (C0907637)

Harmanjot Singh Dhanoa (C0907635)

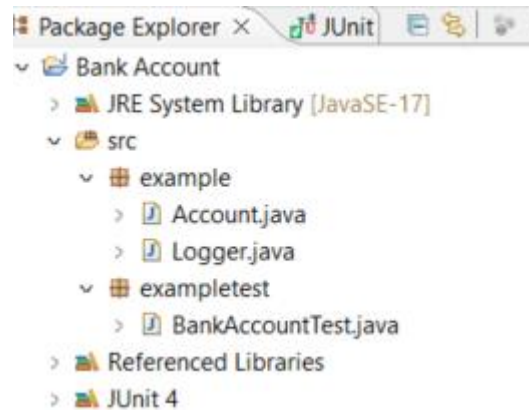
Khushpreet Singh (C0917431)

Karandeep Kaur (C0915617)

Instructor: Alexandre Gorelenko

April 12, 2024

Bank Account Management



Account.java

```
1 package example;
2
3 public class Account {
4     private String accountNumber;
5     private double balance;
6     private Logger logger;
7
8     public Account(String accountNumber, Logger logger) {
9         this.accountNumber = accountNumber;
10        this.balance = 0.0;
11        this.logger = logger;
12    }
13
14    public String getAccountNumber() {
15        return accountNumber;
16    }
17
18    public double getBalance() {
19        return balance;
20    }
21
22    public void deposit(double amount) {
23        if (amount > 0) {
24            balance += amount;
25            //logger.log("Deposit of $" + amount + " to account " + accountNumber);
26        } else {
27            throw new IllegalArgumentException("Deposit amount must be greater than zero");
28        }
29    }
30
31    public void withdraw(double amount) {
32        if (amount > 0 && amount <= balance) {
33            balance -= amount;
34            //logger.log("Withdrawal of $" + amount + " from account " + accountNumber);
35        } else {
36            throw new IllegalArgumentException("Withdrawal amount must be greater than zero or equal to balance");
37        }
38    }
39
40    public void printAccountInfo() {
41        System.out.println("Account Number: " + accountNumber);
42        System.out.println("Balance: $" + balance);
43    }
44 }
45
```

BankAccountTest – JUnit Test

1. testDeposit()

```
11
12
13 public class BankAccountTest {
14     private Account account;
15     //private Logger mockLogger;
16
17     @Before
18     public void setUp() {
19         //mockLogger = Mockito.mock(Logger.class);
20         //account = new Account("123456789", mockLogger);
21         account = new Account("123456789");
22     }
23
24     @After
25     public void tearDown() {
26         //Tear down not needed in this case as we don't have any resources to clean up
27     }
28
29     @Test
30     public void testDeposit() {
31         account.deposit(100.0);
32         assertEquals(100.0, account.getBalance(), 0.001);
33         account.printAccountInfo();
34         //Mockito.verify(mockLogger).log("Deposit of $100.0 to account 123456789");
35     }
36
```

<terminated> BankAccountTest [JUnit] C:\Users\Umesh.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_17.0.9.v20231028-0858\jre\bin\javaw.exe (Apr 12, 2024, 2:04:20 PM)
Account Number: 123456789
Balance: \$100.0

Passed Scenario: Deposit of 100.0 succeeds, and the balance matches.

```
Runs: 6/6 Errors: 0 Failures: 4
exampletest.BankAccountTest [Runner: JUnit 4] (0.024 s)
  testWithdrawInsufficientFunds (0.013 s)
  testWithdrawSufficientFunds (0.002 s)
  testDepositNegativeAmount (0.001 s)
  testDeposit (0.000 s)
  testWithdrawNegativeAmount (0.008 s)
  testWithdrawZeroAmount (0.000 s)

13 public class BankAccountTest {
14     private Account account;
15     //private Logger mockLogger;
16
17     public void setUp() {}
18
19     public void tearDown() {}
20
21     @Test
22     public void testDeposit() {
23         account.deposit(100.0);
24         assertEquals(200.0, account.getBalance(), 0.001);
25         account.printAccountInfo();
26         //Mockito.verify(mockLogger).log("Deposit of $100.0 to account 123456789");
27     }
28
```

Failed: Deposit of 100.0 fails, resulting in a balance mismatch.

2. testWithdrawSufficientFunds()

```
28
29
30 public void testDeposit() {}
31
32 @Test
33 public void testWithdrawSufficientFunds() {
34     account.deposit(100.0);
35     account.withdraw(50.0);
36     assertEquals(50.0, account.getBalance(), 0.001);
37     account.printAccountInfo();
38 }
39
```

<terminated> BankAccountTest [JUnit] C:\Users\Umesh.p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_17.0.9.v20231028-0858\jre\bin\javaw.exe (Apr 12, 2024, 2:04:20 PM)
Account Number: 123456789
Balance: \$100.0
Account Number: 123456789
Balance: \$50.0

Passed: Withdrawal of 50.0 succeeds from an account with a balance of 100.0, leaving 50.0 remaining.

```

16
18* public void setUp() {}
23
25* public void tearDown() {}
28
30* public void testDeposit() {}
36
37* @Test
38* public void testWithdrawSufficientFunds() {
39*     account.deposit(100.0);
40*     account.withdraw(50.0);
41*     assertEquals(60.0, account.getBalance(), 0.001);
42*     account.printAccountInfo();
43* }
44

```

Failed: Withdrawal of 50.0 fails, resulting in an incorrect balance.

3. testWithdrawInsufficientFunds()

```

11
12
13 public class BankAccountTest {
14     private Account account;
15     //private Logger mockLogger;
16
17     @Before
18     public void setUp() {
19         //mockLogger = Mockito.mock(Logger.class);
20         //account = new Account("123456789", mockLogger);
21         account = new Account("123456789");
22     }
23
24     @After
25     public void tearDown() {
26         //Tear down not needed in this case as we don't have any resources to clean up
27     }
28
29     public void testDeposit() {}
30
31     public void testWithdrawSufficientFunds() {}
32
33
34
35
36
37
38
39
40
41
42
43
44
45     @Test
46     public void testWithdrawInsufficientFunds() {
47         account.deposit(100.0);
48         assertThrows(IllegalArgumentException.class, () -> {
49             account.withdraw(150.0);
50         });
51         account.printAccountInfo();
52     }
53

```

Passed:

- Test verifies that attempting to withdraw more money than the account balance results in the correct exception being thrown.
- The test “testWithdrawInsufficientFunds” effectively validates the behavior of the “withdraw” method in handling insufficient funds scenarios which was of the “BankAccount” class.

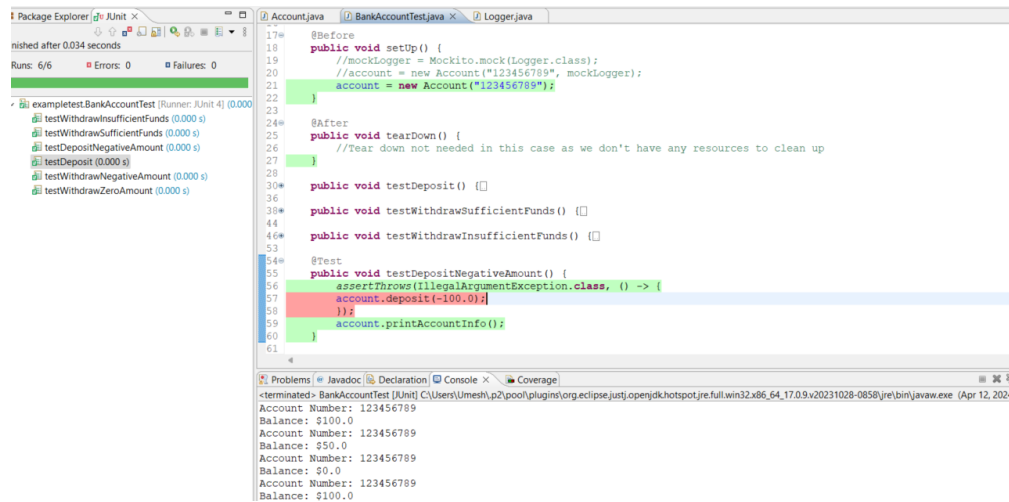
```

44
45     @Test
46     public void testWithdrawInsufficientFunds() {
47         account.deposit(100.0);
48         assertThrows(IllegalArgumentException.class, () -> {
49             account.withdraw(150.0);
50         });
51         account.printAccountInfo();
52     }
53

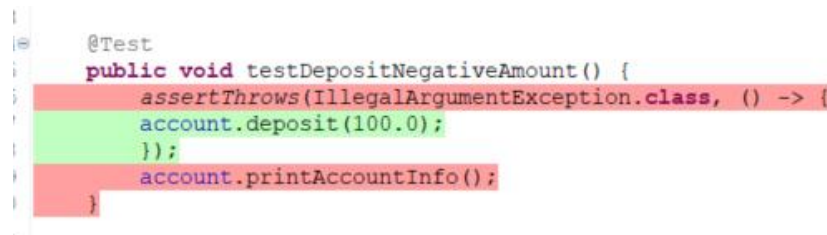
```

Failed: Withdrawal of 150.0 does not throw an *IllegalArgumentException*.

4. testDepositNegativeAmount()

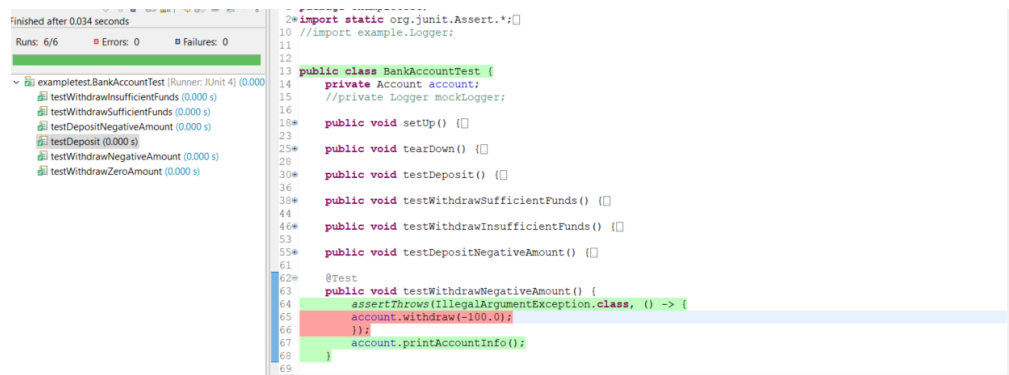


Passed: Attempting to deposit -100.0 correctly **throws** an *IllegalArgumentException*.



Failed: Deposit of -100.0 does not throw an *IllegalArgumentException*.

5. testWithdrawNegativeAmount()



Passed: Attempting to withdraw -100.0 correctly **throws** an *IllegalArgumentException*.

Failed: Withdrawal of -100.0 **does not throw** an *IllegalArgumentException*.

6. testWithdrawZeroAmount()

```
63* public void testWithdrawNegativeAmount() {}
69
70 // edge cases
71 @Test
72 public void testWithdrawZeroAmount() {
73     assertThrows(IllegalArgumentException.class, () -> {
74         account.withdraw(0.0);
75     });
76     account.printAccountInfo();
77 }
78 }
79
80
```

Problems | Javadoc | Declaration | Console | Coverage

terminated> BankAccountTest [JUnit] C:\Users\Umesh\p2\poo\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17

Account Number: 123456789
Balance: \$100.0
Account Number: 123456789
Balance: \$50.0
Account Number: 123456789
Balance: \$0.0
Account Number: 123456789
Balance: \$100.0
Account Number: 123456789
Balance: \$0.0
Account Number: 123456789
Balance: \$0.0

Passed: Attempting to withdraw 0.0 correctly **throws** an *IllegalArgumentException*

Failed: Withdrawal of 0.0 does **not** throw an *IllegalArgumentException*.

Coverage

Passed

Element	Coverage	Covered Instru...	Missed Instruct...	Total Instructio...
BankAccountTest (Apr 12, 2024 2:04:21 PM)				
Bank Account	82.4 %	145	31	176
src	82.4 %	145	31	176
exampletest	80.6 %	83	20	103
BankAccountTest.java	80.6 %	83	20	103
BankAccountTest	80.6 %	83	20	103
setUp()	100.0 %	7	0	7
tearDown()	100.0 %	1	0	1
testDeposit()	100.0 %	14	0	14
testDepositNegativeAmount()	100.0 %	9	0	9
testWithdrawInsufficientFunds()	100.0 %	13	0	13
testWithdrawNegativeAmount()	100.0 %	9	0	9
testWithdrawSufficientFunds()	100.0 %	18	0	18
testWithdrawZeroAmount()	100.0 %	9	0	9

Finished after 0.036 seconds

Runs: 6/6 Errors: 0 Failures: 0

exampletest.BankAccountTest [Runner: JUnit 4] (0.000 s)

- testDeposit (0.000 s)
- testDepositNegativeAmount (0.000 s)
- testWithdrawSufficientFunds (0.000 s)
- testWithdrawInsufficientFunds (0.000 s)
- testWithdrawNegativeAmount (0.000 s)
- testWithdrawZeroAmount (0.000 s)

Failed

Element	Coverage	Covered Instru...	Missed Instruct...	Total Instructio...
BankAccountTest (Apr 12, 2024 2:42:02 PM)				
Bank Account	63.6 %	112	64	176
src	63.6 %	112	64	176
exampletest	53.4 %	55	48	103
BankAccountTest.java	53.4 %	55	48	103
BankAccountTest	53.4 %	55	48	103
testDeposit()	28.6 %	4	10	14
testWithdrawSufficientFunds()	44.4 %	8	10	18
testDepositNegativeAmount()	0.0 %	0	9	9
testWithdrawInsufficientFunds()	30.8 %	4	9	13
setUp()	100.0 %	7	0	7
tearDown()	100.0 %	1	0	1
testWithdrawNegativeAmount()	100.0 %	9	0	9
testWithdrawZeroAmount()	100.0 %	9	0	9

Runs: 6/6 Errors: 0 Failures: 4

exampletest.BankAccountTest [Runner: JUnit 4] (0.024 s)

- testWithdrawInsufficientFunds (0.013 s)
- testWithdrawSufficientFunds (0.002 s)
- testDepositNegativeAmount (0.001 s)
- testDeposit (0.000 s)
- testWithdrawNegativeAmount (0.008 s)
- testWithdrawZeroAmount (0.000 s)

Mockito

Package Explorer

JUnit

Finished after 0.152 seconds

Runs: 6/6 Errors: 6 Failures: 0

exampleTest.BankAccountTest [Runner: JUnit 4] (0.002 s)

testWithdrawSufficientFunds (0.000 s)

testWithdrawSufficientFunds (0.001 s)

testDepositNegativeAmount (0.000 s)

testDeposit (0.000 s)

testWithdrawNegativeAmount (0.000 s)

testWithdrawZeroAmount (0.001 s)

Failure Trace

java.lang.IllegalStateException: Could not initialize plugin: interface org.mockito.plugins.S

at org.mockito.internal.configuration.plugins.PluginLoader\$1.invoke(PluginLoader.java:85)

atjdk.proxy2/jdk.proxy2\$Proxy11.isTypeMockable(Unknown Source)

at org.mockito.internal.util.MockUtil.typeMockabilityOf(MockUtil.java:78)

at org.mockito.internal.util.MockCreationValidator.validateType(MockCreationValidator.java:11)

at org.mockito.internal.creation.MockSettingsImpl.validateSettings(MockSettingsImpl.java:236)

at org.mockito.internal.MockCore.mock(MockCore.java:82)

at org.mockito.Mockito.mock(Mockito.java:2104)

at org.mockito.Mockito.mock(Mockito.java:2019)

at exampleTest.BankAccountTest.setUp(BankAccountTest.java:19)

Caused by: java.lang.IllegalStateException: Internal problem occurred, please report it. Mc

at org.mockito.internal.configuration.plugins.DefaultMockitoPlugins.create(DefaultMockito

at org.mockito.internal.configuration.plugins.DefaultMockitoPlugins.getDefaultPlugin(De

at org.mockito.internal.configuration.plugins.PluginLoader.loadPlugin(PluginLoader.java:7

at org.mockito.internal.configuration.plugins.PluginLoader.loadPlugin(PluginLoader.java:7)

Account.java

BankAccountTest.java X

Logger.java

```
1 package exampleTest;
2 import static org.junit.Assert.*;
3 import org.junit.Before;
4 import org.junit.After;
5 import org.junit.Test;
6 import org.mockito.Mockito;
7 import org.mockito.plugins.MockMaker;
8
9 import example.Account;
10 import example.Logger;
11
12
13 public class BankAccountTest {
14     private Account account;
15     private Logger mockLogger;
16
17     @Before
18     public void setUp() {
19         mockLogger = Mockito.mock(Logger.class);
20         account = new Account("123456789", mockLogger);
21         //account = new Account("123456789");
22     }
23
24     @After
25     public void tearDown() {
26         //Tear down not needed in this case as we don't have any resources to clean up
27     }
28
29     @Test
30     public void testDeposit() {
31         account.deposit(100.0);
32         assertEquals(100.0, account.getBalance(), 0.001);
33         account.printAccountInfo();
34         Mockito.verify(mockLogger).log("Deposit of $100.0 to account 123456789");
35     }
36
37     @Test
38     public void testWithdrawSufficientFunds() {
39         account.deposit(100.0);
40         account.withdraw(50.0);
41         assertEquals(50.0, account.getBalance(), 0.001);
42         account.printAccountInfo();
43     }
44
45     @Test
46     public void testWithdrawInsufficientFunds() {
47         account.deposit(100.0);
48         assertThrows(IllegalArgumentException.class, () -> {
49             account.withdraw(150.0);
50         });
51     }
52 }
```

Problems

Javadoc

Declaration

Console

Coverage X

BankAccountTest (Apr 12, 2024 3:16:03 PM)

Element	Coverage	Covered Instru...	Missed Instruct...	Total Instructio...
Bank Account	1.9 %	4	202	206
src	1.9 %	4	202	206
exampleTest	3.4 %	4	112	116
BankAccountTest.java	3.4 %	4	112	116
example	0.0 %	0	90	90